

OES-32 Telemetry Triage Engine

Technical Specification

Document identifier: OES32-TS-001

Revision: 1.0

Status: Reference software specification

Date: 28 August 2026

Author: Jean-François Brisson

Organization: Spark AI NLP

Repository: [sparkainlp-x/oes32_engine](#)

Safety and validation notice. This document specifies a deterministic software reference implementation and test harness. It is not a certification artifact, a hardware design approval, or an authorization to connect the implementation directly to safety-critical equipment. Any operational deployment requires independent requirements review, numerical analysis, hardware-in-the-loop testing, fail-safe design review, cybersecurity review, and formal verification appropriate to the system's hazard classification.

1. Purpose and scope

The OES-32 telemetry triage engine evaluates a proposed 32-element state vector against a reference baseline and applies three classes of consistency checks: maximum residual coherence, bipartite sector symmetry, and continuity across four cyclic eight-node rings. The engine reduces these measurements to a deterministic containment decision. A state is accepted only when every configured constraint is within its threshold; otherwise the result is marked latched.

The scope of this specification is the pure computational behavior represented by the Python module `oes32_engine.py`. It covers data contracts, indexing, equations, threshold semantics, validation, result reporting, error behavior, test expectations, numerical considerations, and integration boundaries. It does not define telemetry transport, sensor calibration, hardware latch circuitry, real-time scheduling, persistence, authentication, or a certified safety case.

2. Design principles

The implementation follows five principles. First, it is deterministic: identical finite inputs and thresholds produce identical outputs. Second, it is conservative at the aggregate decision

boundary: all predicates must pass for `safe=True` . Third, a strict exceedance is distinguished from equality, so a residual exactly equal to a threshold passes that individual check. Fourth, all vector dimensions are explicit and fixed at 32. Fifth, unspecified domain conventions are surfaced as configuration rather than silently hidden.

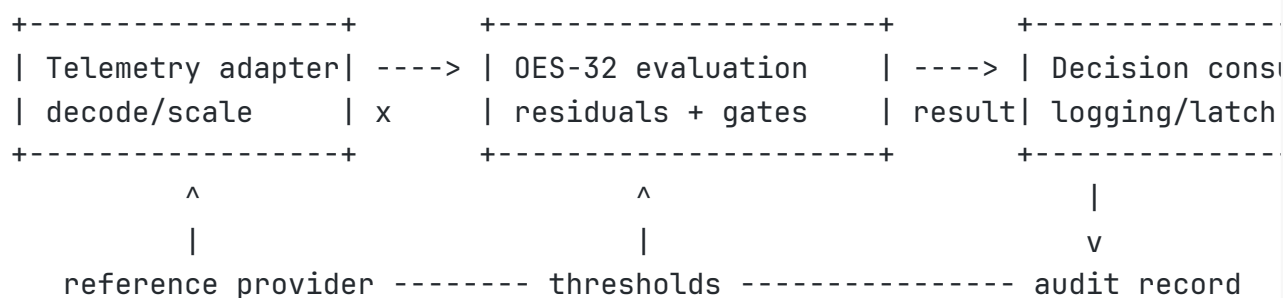
Principle	Specification consequence
Determinism	No random state, I/O, clock, or mutable global evaluation state is required.
Fixed topology	Every state and reference vector contains exactly 32 finite numeric elements.
Max-residual detection	A single worst element is sufficient to fail a metric.
Fail-closed aggregation	Any failed metric produces <code>safe=False</code> and <code>latch=True</code> .
Explicit ambiguity handling	The odd-sector sign convention is configurable and documented.

3. Terminology and symbols

Symbol or term	Meaning
<code>W</code>	State-vector width; fixed at 32.
<code>x</code>	Proposed or observed state vector.
<code>x_ref</code>	Reference baseline vector.
<code>i</code>	Global vector index, from 0 through 31.
<code>mu(i)</code>	Bipartite partner mapping <code>(i + 16) mod 32</code> .
<code>tau</code>	Primary coherence threshold; default <code>0.08</code> .
<code>tau_sym</code>	Sector symmetry threshold; defaults to <code>tau</code> when omitted.
<code>tau_fold</code>	FOLD8 continuity threshold; defaults to <code>tau</code> when omitted.
<code>R</code>	Maximum absolute deviation from the reference baseline.
<code>S_0</code> , <code>S_1</code>	EVEN and ODD sector symmetry residuals.
<code>F</code>	Maximum FOLD8 ring continuity residual.
<code>safe</code>	Aggregate acceptance predicate.
<code>latch</code>	Aggregate containment predicate; the logical inverse of <code>safe</code> .

4. System boundary and logical data flow

The engine is intentionally a pure evaluation layer. An upstream adapter is responsible for acquiring, decoding, timestamping, scaling, and plausibility-checking telemetry. The engine receives already-decoded numeric sequences. A downstream controller may consume the returned metrics, but the library itself does not actuate a physical device.



The reference provider must use the same coordinate ordering, units, calibration state, and epoch assumptions as the proposed vector. The engine cannot infer or correct a mismatch in those external assumptions.

5. Mathematical specification

5.1 State space and partner mapping

Let $[W=32, \sqcup x, x_{\text{ref}} \in \mathbb{R}^{32}, \sqcup i \in \{0, 1, \dots, 31\}]$. The partner mapping is $[\mu(i) = (i+16) \bmod 32]$. This mapping is an involution: applying it twice returns the original index, $\mu(\mu(i)) = i$. The implementation computes the mapping directly and does not allocate a separate topology table.

5.2 Residual coherence

The primary residual is the infinity-norm distance between the proposed and reference vectors: $[R(x, x_{\text{ref}}) = \max_{0 \leq i < W} |x_i - x_{\text{ref}, i}|]$. The corresponding acceptance predicate is $[A = \mathbf{1}[R \leq \tau], \sqcup \tau = 0.08 \text{ by default}]$. The maximum operation makes the check sensitive to the worst element and insensitive to the number of elements that remain near the baseline.

5.3 Circuit-breaker latch

The circuit-breaker state is defined as $[L = \mathbf{1}[R > \tau]]$. At the primary residual level, $R = \tau$ is accepted and $R > \tau$ is rejected. The aggregate implementation additionally treats any failed symmetry or FOLD8 predicate as a latch condition: $[\text{SAFE} = A \text{ and } C_0 \text{ and } C_1 \text{ and } C_{\text{fold}}, \sqcup \text{LATCH} = \neg \text{SAFE}]$. Therefore, the returned `latch` field is true whenever any configured metric fails, not only when the primary residual fails.

5.4 EVEN/ODD sector symmetry

The partner mapping preserves parity because 16 is even. For sector parity p in $\{0, 1\}$, the engine evaluates $[S_p = \max_{i \equiv p \pmod 2} |x_i - q_i x^{\mu(i)}|]$. The reference convention is $q_i = +1$ for EVEN indices and $q_i = -1$ for ODD indices. Equivalently: $[S_0 = \max_{i \text{ even}} |x_i - x^{\mu(i)}|, \quad S_1 = \max_{i \text{ odd}} |x_i + x^{\mu(i)}|]$. Each sector passes when $[C_p = \mathbf{1}[S_p \leq \tau_{\text{sym}}]]$. The original domain prompt named EVEN/ODD constraints but did not provide the expected sign relation. Accordingly, the implementation exposes `antisymmetric_odd=True` as the explicit default. Setting it to false uses equality for both sectors. This parameter must be fixed and versioned by any integrating system; it should not vary between evaluations in the same control regime.

5.5 FOLD8 ring continuity

The 32-element spine is partitioned into four rings, each with eight nodes. Ring s has nodes k in $\{0, \dots, 7\}$ and flattened index $j = 8s + k$. The cyclic successor is $(k+1) \bmod 8$. The continuity residual is $[F = \max_{0 \leq s < 4} \max_{0 \leq k < 8} |x_{8s+k} - x_{8s+(k+1) \bmod 8}|]$. The continuity predicate is $[C_{\text{fold}} = \mathbf{1}[F \leq \tau_{\text{fold}}]]$. The modulo operation includes the closing edge from node 7 back to node 0 within every ring.

6. Software interface

The public interface is exposed by the package-level module:

```
from oes32_engine import Evaluation, evaluate, fold8_residual, residual, ...
```

6.1 Constants

Name	Type	Value	Meaning
<code>WIDTH</code>	<code>int</code>	<code>32</code>	Required vector width.
<code>MU_OFFSET</code>	<code>int</code>	<code>16</code>	Partner-map offset.
<code>DEFAULT_TAU</code>	<code>float</code>	<code>0.08</code>	Default primary threshold.

6.2 Functions

Function	Inputs	Output	Contract
<code>residual(x, x_ref)</code>	Two 32-element sequences	<code>float</code>	Returns <code>R</code> .
<code>symmetry_residual(x, parity, antisymmetric_odd=True)</code>	State, parity 0/1, sign mode	<code>float</code>	Returns <code>S_0</code> or <code>S_1</code> .
<code>fold8_residual(x)</code>	One 32-element sequence	<code>float</code>	Returns <code>F</code> .
<code>evaluate(x, x_ref, tau=0.08, tau_sym=None, tau_fold=None, antisymmetric_odd=True)</code>	Vectors and thresholds	<code>Evaluation</code>	Computes all metrics and aggregate decision.

6.3 Evaluation result

`Evaluation` is an immutable dataclass with the following fields:

Field	Type	Meaning
<code>residual</code>	<code>float</code>	Primary <code>R</code> .
<code>latch</code>	<code>bool</code>	True when aggregate acceptance fails.
<code>even_symmetry_residual</code>	<code>float</code>	EVEN <code>S_0</code> .
<code>odd_symmetry_residual</code>	<code>float</code>	ODD <code>S_1</code> .
<code>fold8_residual</code>	<code>float</code>	FOLD8 <code>F</code> .
<code>safe</code>	<code>bool</code>	True only when all predicates pass.

A consumer should record all residual fields rather than only the Boolean decision. The metrics are needed for diagnosis, threshold review, replay, and auditability.

7. Input validation and error behavior

Each vector must have exactly 32 elements. Values are converted to `float` and must be finite; `NaN` , positive infinity, and negative infinity are rejected. Thresholds must be non-negative. `parity` must be either 0 or 1. Violations raise `ValueError` before a decision result is returned.

The implementation does not impose an application-specific physical range because the valid range depends on the telemetry channel and calibration contract. Range checks, unit checks, freshness checks, sequence checks, and sensor-quality flags belong in the upstream adapter or a separate validation layer.

Invalid condition	Required behavior
Vector length is not 32	Raise <code>ValueError</code> .
Vector contains a non-finite value	Raise <code>ValueError</code> .
Any threshold is negative	Raise <code>ValueError</code> .
<code>parity</code> is not 0 or 1	Raise <code>ValueError</code> .
Valid vectors and thresholds	Return complete <code>Evaluation</code> .

8. Decision semantics and examples

The following examples use the default threshold `0.08` and a zero reference vector. They demonstrate boundary behavior, not physical operating limits.

Scenario	Primary observation	Expected result
All-zero proposed and reference vectors	<code>R=0</code> , <code>S_0=0</code> , <code>S_1=0</code> , <code>F=0</code>	<code>safe=True</code> , <code>latch=False</code> .
One element equals <code>0.08</code> with all other constraints passing	<code>R=tau</code>	Primary coherence passes because the comparison is non-strict.
One element equals <code>0.080001</code>	<code>R>tau</code>	<code>latch=True</code> .
A ring closing edge differs by <code>0.20</code>	<code>F=0.20</code>	FOLD8 fails and aggregate result latches.
Vector width is 31	Invalid input	<code>ValueError</code> ; no decision is emitted.

A key integration rule is that a failed input validation is not equivalent to a safe state. The caller must route exceptions into its own fault-handling path and must not treat the absence of an `Evaluation` object as acceptance.

9. Numerical and implementation considerations

The implementation uses ordinary binary floating-point conversion and absolute differences. Threshold comparisons should therefore be tested with representative values near the boundary, including values produced by the actual telemetry decoder. If a system requires decimal-exact behavior, fixed-point or a formally specified tolerance policy should be introduced at the system boundary rather than assumed by this reference module.

The algorithms are linear in the vector width. Residual coherence examines 32 differences, each sector examines 16 partner comparisons, and FOLD8 examines 32 cyclic edges. With the fixed topology, the computational work is bounded and small; nevertheless, an operational system should measure execution time, scheduling jitter, memory behavior, and failure recovery on its target platform.

Operation	Comparisons	Space characteristic
Primary residual	32	Constant auxiliary storage.
EVEN symmetry	16	Constant auxiliary storage.
ODD symmetry	16	Constant auxiliary storage.
FOLD8 continuity	32	Constant auxiliary storage.
Aggregate evaluation	Sum of above	Returns six scalar fields.

10. Verification strategy

The repository includes a standard-library `unittest` suite. The current tests cover zero-state acceptance, maximum absolute residual selection, threshold equality, strict exceedance, FOLD8 wraparound, the documented symmetry convention, and invalid vector width. These tests establish basic functional behavior but are not a complete verification or safety validation campaign.

Recommended additional verification includes property-based tests for the partner-map involution, permutation tests proving that primary residual results do not depend on element order, exhaustive boundary tests around each threshold, non-finite input tests, independent implementation cross-checks, mutation testing, static analysis, timing measurements on the deployment platform, and hardware-in-the-loop tests with a separately reviewed oracle.

11. Integration and operational requirements

An integrating system should pin the implementation revision, record the threshold set and sign convention with every decision, retain the full residual vector or sufficient diagnostic evidence, and define a recovery policy outside this library. The integration should also define what happens when telemetry is stale, missing, duplicated, out of order, malformed, or inconsistent with the reference baseline's coordinate system.

The downstream consumer must treat `latch=True` as a request for its configured containment path, not as proof that a particular physical fault has been identified. The engine detects mathematical inconsistency against the supplied rules; it does not diagnose root cause.

12. Traceability matrix

Requirement identifier	Requirement	Implementation location	Verification
OES32-REQ-001	Accept only 32-element vectors	<code>_validate_vector</code>	Invalid-width test.
OES32-REQ-002	Reject non-finite values	<code>_validate_vector</code>	Recommended extension test.
OES32-REQ-003	Compute max absolute baseline deviation	<code>residual</code>	Residual test.
OES32-REQ-004	Use strict exceedance for primary latch threshold	<code>evaluate</code>	Equality and exceedance tests.
OES32-REQ-005	Apply parity partner mapping with offset 16	<code>symmetry_residual</code>	Symmetry fixture test.
OES32-REQ-006	Evaluate cyclic FOLD8 closing edges	<code>fold8_residual</code>	Wraparound test.
OES32-REQ-007	Require all gates for aggregate safety	<code>evaluate</code>	Zero-state and failure tests.
OES32-REQ-008	Expose diagnostic residuals	<code>Evaluation</code>	Result dataclass inspection.

13. Limitations, assumptions, and open decisions

The sign convention for the ODD sector was not fully specified in the originating requirements and is therefore an explicit configuration option. The current reference default is antisymmetric ODD behavior. The specification also assumes that the 32-element vector is already normalized into a common numerical space; it does not define channel-specific units or weights. All thresholds are scalar absolute tolerances, with no hysteresis, debounce, temporal voting, or persistence requirement.

Before any production or safety-related use, the owner should decide whether the containment decision must include temporal filtering, threshold hysteresis, independent sensor voting, stale-

data rejection, reason codes, event sequencing, or a separate hardware interlock. Those decisions must be added as versioned requirements and tested independently.

14. Change control and reproducibility

The repository revision, Python version, threshold values, `antisymmetric_odd` setting, input ordering, reference-vector provenance, and test command should be recorded for every qualification run. Changes to equations, index mapping, threshold semantics, or result fields require a specification revision and regression-test update.

15. License and attribution

Released under the GNU Affero General Public License v3.0 only (AGPL-3.0-only). A commercial license is available; see `COMMERCIAL-LICENSE.md`. Versions published before 2026-09-29 were released under the MIT License and remain available under those terms. The canonical license text is maintained in the repository-root `LICENSE` file. The project is authored by Jean-François Brisson for Spark AI NLP. The license badge, README statement, source-file SPDX notices, this specification, and the repository-root license are intended to remain consistent.

References